

Université de Sherbrooke
Département d'informatique
IGL501/IGL710 : Méthodes formelles en génie logiciel
Devoir 4

Date de remise : vendredi 28 novembre à 16:00

En équipe de 3 ou 4. À remettre avec Turninweb (<http://turnin.dinf.usherbrooke.ca/>)

Fichiers à remettre :

- q1.als, q2.als, q3.als, q4.als

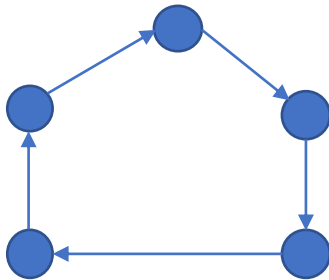
Note : utilisez Alloy5 (et non pas Alloy6)

<https://github.com/AlloyTools/org.alloytools.alloy/releases/tag/v5.1.0>

- MAC OSX : utilisez
<https://github.com/AlloyTools/org.alloytools.alloy/releases/download/v5.1.0/alloy.dmg>
- Windows, Linux : Utilisez ce .jar, qui démarre Alloy lorsque vous le lancez.
<https://github.com/AlloyTools/org.alloytools.alloy/releases/download/v5.1.0/org.alloytools.alloy.dist.jar>

Question 1

Écrivez une spécification en Alloy d'un anneau. Un anneau est un graphe cyclique qui comprend un seul cycle qui contient tous les nœuds. Complétez la spécification q1.als



Question 2

Écrivez une spécification en Alloy d'un arbre binaire, c'est-à-dire que chaque nœud a au plus 2 fils. Complétez la spécification q2.als

Question 3

a)

Spécifiez en Alloy le système de contrôle d'accès suivant. Le système est composé de règles, d'un graphe orienté acyclique de sujets et d'un graphe orienté acyclique de ressources. Dans un

graphe de sujets ou de ressources, on dit que a' est un ancêtre de a s'il existe un chemin de a vers a' . On dit que a est plus spécifique que a' ssi si a' est un ancêtre de a .

L'algorithme qui calcule l'autorisation repose sur les principes suivants.

- Chaque règle porte sur un sujet (celui qui émet la requête), une ressource (les données visées par la requête), une priorité et une modalité (permission ou interdiction).
- Une règle L s'applique à une requête (s,r) ssi
 - L .sujet est un ancêtre de s dans le graphe des sujets
 - L .ressource est ancêtre de r dans le graphe des sujets
- Les règles applicables sont ordonnées de la manière suivante (noté $L1 \ll L2$, et appelée « $L1$ précède $L2$ »):
 - Les règles les plus prioritaires ont précedence (ie, si $L1.priorité < L2.priorité$, alors $L1 \ll L2$.,
 - Si $L1$ et $L2$ ont la même priorité, alors $L1 \ll L2$ si $L2.sujet$ est un ancêtre de $L1.sujet$.
- Si aucune règle ne s'applique à une requête, alors l'accès est interdit.
- Si plusieurs s'appliquent à une requête, alors l'accès est accordé ssi il n'y a aucune interdiction parmi les règles *minimales* de la relation $\ll$. Une règle $L1$ est minimale ssi il n'existe pas de règle $L2$ telle que $L2 \ll L1$.

La Figure 1 illustre quelques exemples simples de règles. À gauche en bleu, on retrouve les sujets. À droite, on retrouve les ressources en jaune. Les flèches rouge ou verte illustrent les règles : le vert dénote une permission, le rouge une interdiction. Pour simplifier, on suppose que toutes les règles ont la même priorité (sinon, l'ordonnancement des règles est très simple).

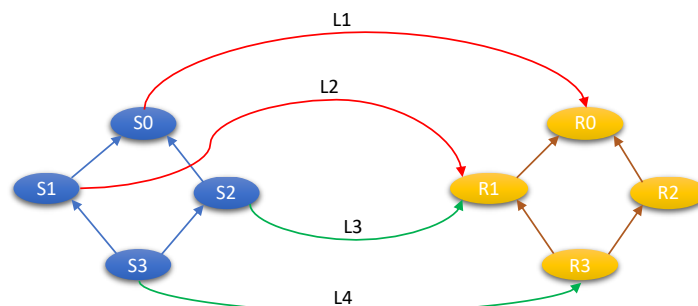


Figure 1

La règle $L1$ a la plus faible précedence, car $L2$, $L3$ et $L4$ sont plus spécifiques que $L1$. La règle $L2$ et la règle $L3$ ne sont pas comparables avec $\ll$, car l'une n'est pas plus spécifique que l'autre. La règle $L4$ a précedence sur toutes les autres règles, car elle est la plus spécifique. Si $S3$ veut accéder à $R3$, alors l'accès est accordé.

Supposons que l'on supprime la règle $L4$. Si $S3$ veut accéder à $R3$, alors l'accès est refusé, car $L2$ et $L3$ ne sont pas comparables avec $\ll$, et vu que $L2$ est une interdiction, alors l'accès est refusée.

b) Codez la fonction

accessible[]: Sujet->Ressource,

qui retourne toutes les paires (s,r) telles que s a accès a r . Par exemple, pour la figure 1, cette fonction retourne $\{S2 \rightarrow R1, S2 \rightarrow R3, S3 \rightarrow R3\}$

c) Codez la fonction

precedence[]: Regle->Regle,

qui retourne toutes les paires $(L1,L2)$ telles $L1 \ll L2$ et $L2$ est le successeur immédiat de $L1$ (ie, $(L1,L2)$ est une arête du graphe des règles).

d) Vérifiez que la relation $\ll$ est

- Acyclique (nommez cette propriété *precedeRelAcyclic*)
- Transitive (nommez cette propriété *precedeRelTransitive*)
- Irreflexive (nommez cette propriété *precedenRelIrreflexive*)

Question 4

Spécifiez en Alloy un algorithme d'acquisition de ressources partagées entre des processus et inspiré de celui de Dijkstra. On a un ensemble de processus P et un ensemble de ressources R . Un processus peut acquérir une ressource si elle est libre. Une ressource est libre si elle n'est pas acquise par un processus. Un processus peut libérer une ressource qu'il a acquise. Dans l'état initial, chaque processus décide de l'ensemble des ressources qu'il veut acquérir. Ensuite, chaque processus acquiert ses ressources, effectue son travail, et libère ensuite ses ressources. Sans contrainte, ce système peut arriver à une impasse (communément appelé *deadlock* en anglais), c'est-à-dire que les processus ne peuvent plus progresser dans l'acquisition des ressources, car ils se bloquent mutuellement. Par exemple, si les processus $P1$ et $P2$ veulent chacun acquérir les ressources $R1$ et $R2$, et que les actions suivantes sont exécutées

Acquerir($P1,R1$); Acquerir($P2,R2$)

alors on est ensuite dans une impasse, car $P1$ veut acquérir $R2$, mais il ne peut pas, car $P2$ l'a déjà acquise, et $P2$ veut acquérir $R1$, mais il ne peut pas, car $P1$ l'a déjà acquise. Pour éviter ces impasses, il suffit d'imposer un ordre strict dans l'acquisition des ressources, et tous les processus doivent suivre cet ordre. Par exemple si l'ordre est $R1 < R2$, alors le premier qui acquiert $R1$ pourra ensuite acquérir $R2$.; l'autre processus devra attendre que le premier libère ses ressources.

Voici les actions du système à spécifier en Alloy

- Init : Associe à chaque processus un ensemble non-vide de ressources à acquérir
- Acquerir($p : P, r : R$) : le processus p acquiert la ressource R si elle est libre.
- Libérer($p : P$) : le processus p libère toutes les ressources qu'il acquise, seulement lorsqu'il a acquis toutes les ressources identifiées dans l'état initial pour ce processus.
- Stutter() : les processus ont atteint l'état final, où tous les processus ont pu acquérir et ensuite libérer leurs ressources, et le système ne fait plus rien. Cette action vous permettra

de créer des traces de longueur fixe avec un nombre variables de ressources à acquérir pour chaque processus.

Écrivez des commandes pour vérifier les éléments suivants

- `deadlock_free` : Chaque processus peut terminer en libérant toutes ses ressources (ie, il n'y a pas d'impasse).
- `show_trace` : Il existe au moins une trace qui permet d'acquérir toutes les ressources pour chaque processus
- Invariant : Une ressource n'est jamais acquise par plus d'un processus à la fois.